# CS 342 Operating Systems

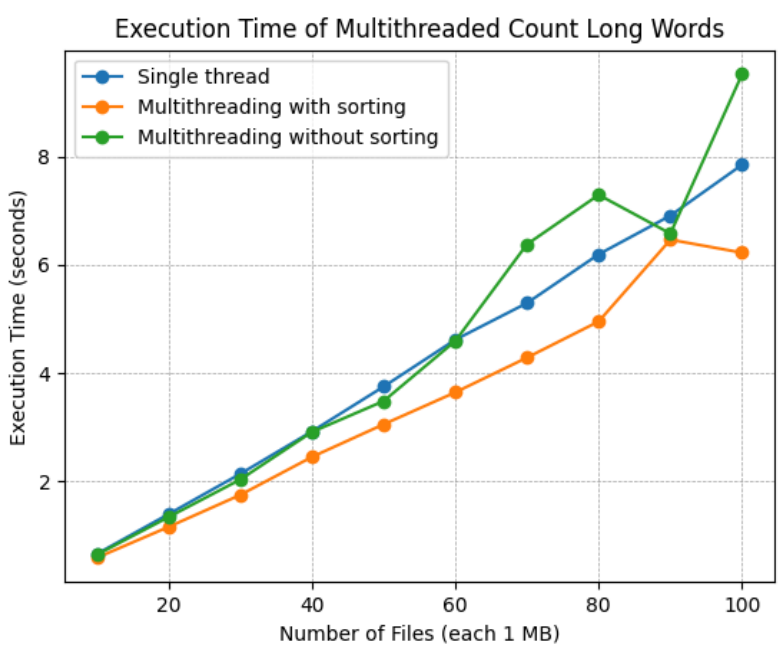
Ömer Can Baykara - 22302436

Project 1

In my submission I could not upload the whole test file directories as they were too large. Instead I uploaded one file in each directory, all other files in the directory were the exact same.

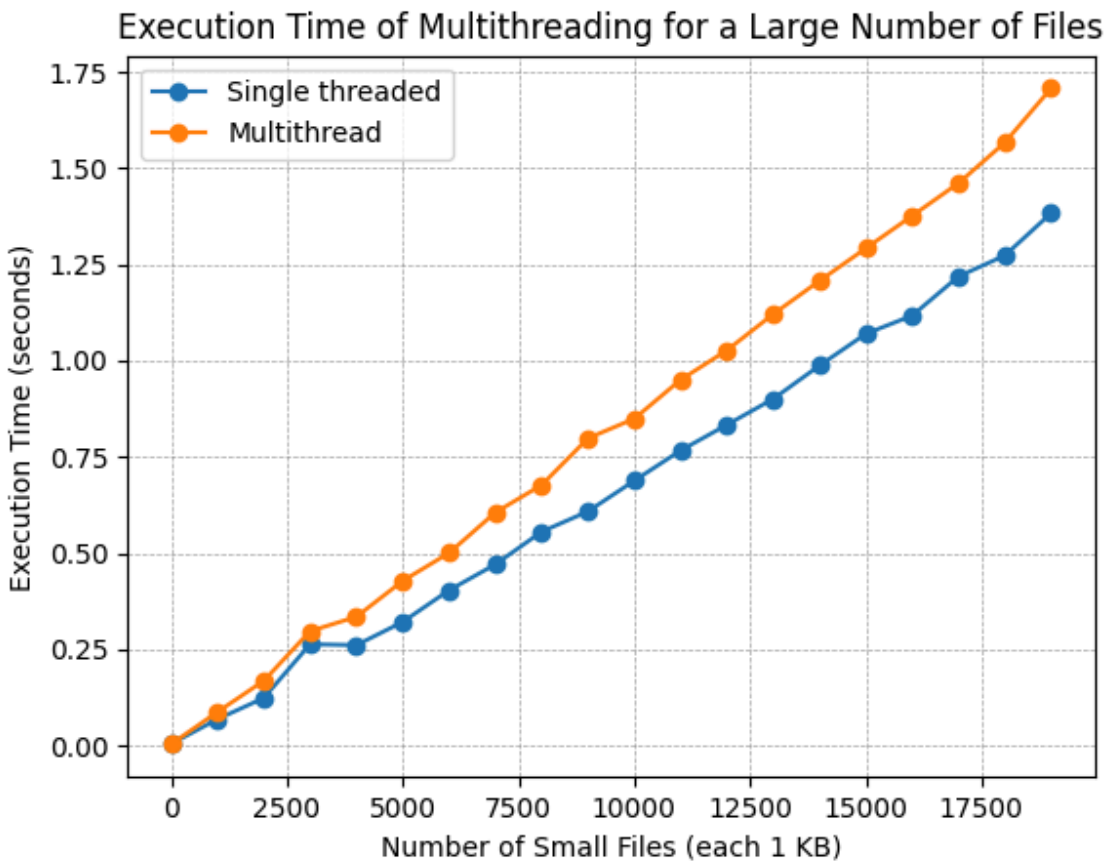
## Experiment 1 (Multithreading)

The experiment was about looking at runtimes of the multithreaded findlwt. I also created a single threaded version which just counted the long words regularly. There were also two versions of findlwt, one where the threads didn't sort their long words before merging, and one where the threads sorted their lists. My prediction was that when the threads didn't sort, the runtime would be slower than the single threaded because of the additional cost of creating threads. Without sorting there wouldn't really be any parallelism as the threads could not do IO in parallel because they are in the same process. When the threads did their own sorting however there would be some data parallelism in the sorting. In the experiment I varied the number of files and measured execution time, keeping k at 0. In the results see the predictions reflected accurately. The non-sorting multithreaded runtime we can see becomes slower than the single threaded runtime as the number of files increases, while the sorting multithreaded programs becomes faster. In conclusion, when doing multithreading it is important to recognize that IO won't be faster and that a speedup can be achieved only by delegating tasks smartly across the threads.



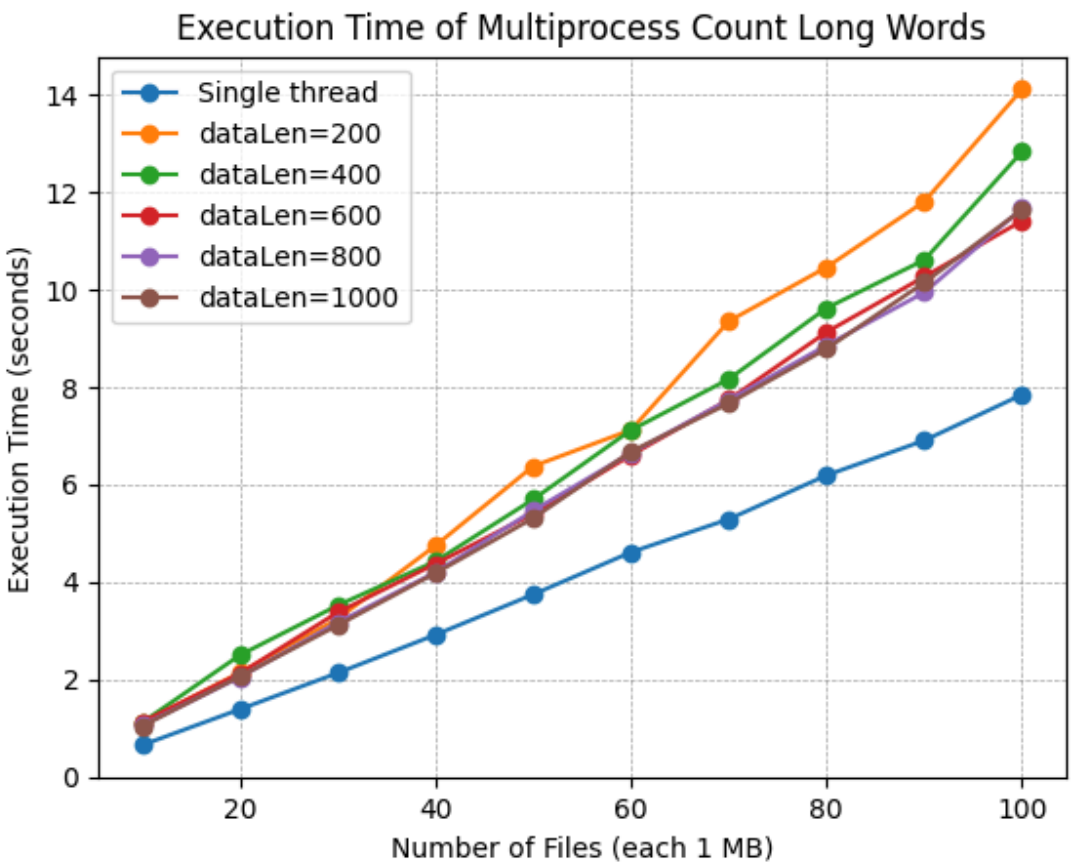
# Experiment 2 (Multithreading for Large N)

In the second experiment I wanted to know whether the relation seen above would continue as the number of files gets really large. For the experiment k was again 0. I predicted that over a certain number, the operating system spends more time context switching because it tries to schedule each thread in a way that it gets a fair slice of the cpu time. So for a really large number context switches have to happen more often. For this reason I predicted that as the number of files got really large the single threaded program would run faster than the multithreaded one (here the multithreaded program is multithreading with sorting). The results show that starting at 1000 files (or threads) the single threaded is faster than the multithreaded and the difference continues to widen. In conclusion, it can be said that cases of data parallelism like we try to do here should be bounded above by a certain limit of threads and should not expand arbitrarily to the input size.



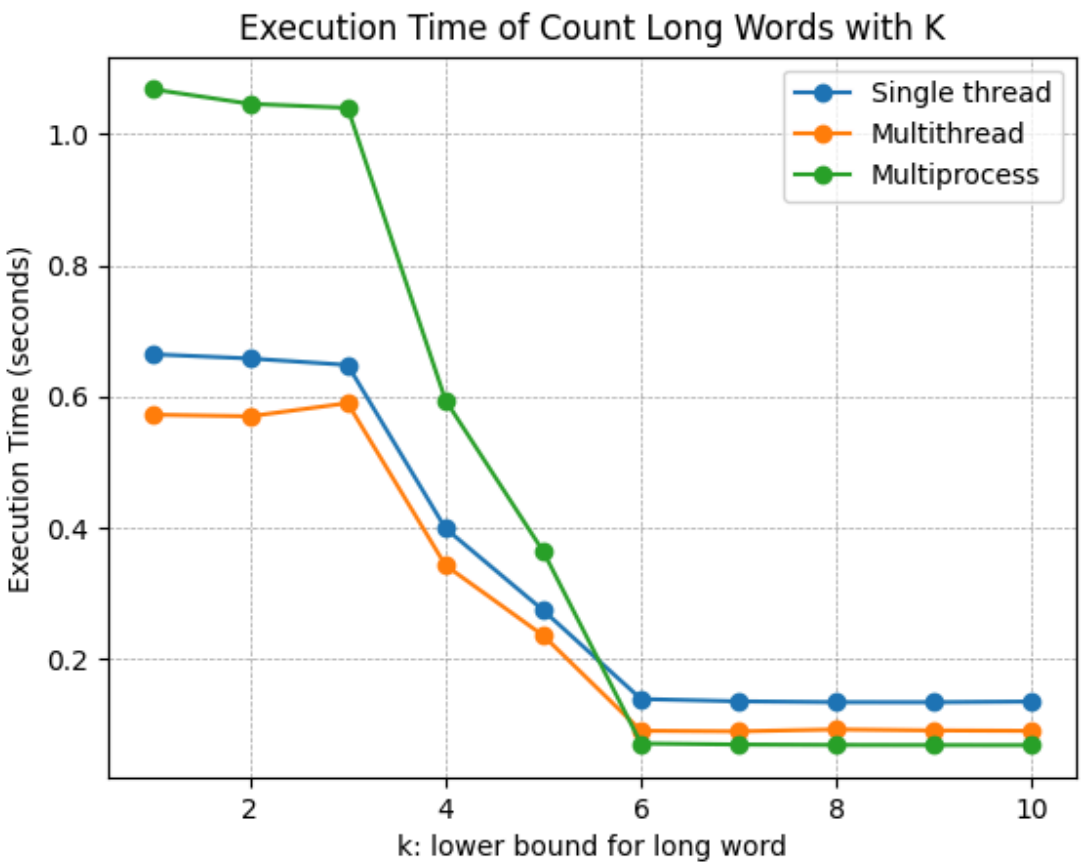
# Experiment 3 (Multiprocessing)

The third experiment involved comparing the execution times of the multiprocess program for varying sizes of data length. K was also 0 in this experiment. Generally with the multiprocess application the bound on the running time was the pipe IO, as each transfer would have to give control back to the kernel. So making the data length larger would make it so less transfers would have to happen thus making it program run faster. So I predicted that as data length increased the execution time would decrease. But then I also thought that no matter the data length it would always be slower than the single threaded program because even though the child processes could do the IO in parallel, there would always be the bottleneck of the parent having to read from all the pipes. It may be hard for the parent to keep up with reading from the pipes. The results show that this is in fact the case and that even though the multiprocess program gets faster as data length increases it never comes close to the runtime of the single threaded one. In conclusion, when using multiple processes, where the program might have a bottleneck is an important consideration and should be thought of.



# Experiment 4 (Varying K)

This experiment involved varying k, the lower bound for long words for the single threaded, multithreaded, and multiprocess programs. For this 50 files were used each of 1 MB, and the dataLen was 1000. I predicted that as k increased the execution time would drop significantly for the multiprocess program as it would have to transfer less data over pipes, which was its bottleneck and for this low k, we could see the benefit of multiprocessing. The results showed this. We see that the execution time decreases with k, but the multiprocess decreases most significantly. This is most probably because the program takes advantage of doing IO in parallel in this case, so it can process the files faster than the single or multithreaded programs, but it does not have to slow itself down by transmitting everything via pipes. So it becomes fastest when the data that has to be transmitted is really small and most of the work is in processing and finding the longest words to transfer. In conclusion, when using IPC, transferring the smallest amount of data can be an important optimization to do for a speedup.



# Tables

## Experiment 1 (Multithreading)

| n | singleThread | notSorting | sorting |
|---|---|---|---|
| 10 | 0.6545517445 | 0.6405916214 | 0.5844659805 |
| 20 | 1.390777349 | 1.335641146 | 1.154604435 |
| 30 | 2.135097027 | 2.029901743 | 1.745363951 |
| 40 | 2.919139147 | 2.908115625 | 2.447593689 |
| 50 | 3.747861147 | 3.480213404 | 3.051028252 |
| 60 | 4.614469051 | 4.590191841 | 3.641905785 |
| 70 | 5.292501688 | 6.378756523 | 4.283825397 |
| 80 | 6.193163395 | 7.291232586 | 4.952069283 |
| 90 | 6.908390045 | 6.576209068 | 6.463100672 |
| 100 | 7.848924398 | 9.523174286 | 6.222958088 |

# Experiment 2 (Multithreading for Large N)

| n | time | threadTime |
|---|---|---|
| 1 | 0.004057407379 | 0.003532648087 |
| 1001 | 0.06830978394 | 0.08710360527 |
| 2001 | 0.1234359741 | 0.1680264473 |
| 3001 | 0.2631387711 | 0.2971453667 |
| 4001 | 0.2607655525 | 0.3345017433 |
| 5001 | 0.3210721016 | 0.4277398586 |
| 6001 | 0.4033684731 | 0.5003087521 |
| 7001 | 0.4706177711 | 0.6054983139 |
| 8001 | 0.5554990768 | 0.6769640446 |
| 9001 | 0.6083748341 | 0.7982828617 |
| 10001 | 0.6880521774 | 0.8493659496 |
| 11001 | 0.76658535 | 0.9502818584 |
| 12001 | 0.8334145546 | 1.027022362 |
| 13001 | 0.9007675648 | 1.121064425 |
| 14001 | 0.9874944687 | 1.207588673 |
| 15001 | 1.069876671 | 1.292109489 |
| 16001 | 1.117267609 | 1.376630545 |
| 17001 | 1.218943119 | 1.461395741 |
| 18001 | 1.27334094 | 1.566798687 |
| 19001 | 1.383497 | 1.708175898 |

# Experiment 3 (Multiprocessing)

| dataLen | n | processTime | dataLen | n | processTime |
|---|---|---|---|---|---|
| 200 | 10 | 1.112489223 | 600 | 60 | 6.606451511 |
| 200 | 20 | 2.141760111 | 600 | 70 | 7.745836258 |
| 200 | 30 | 3.246170282 | 600 | 80 | 9.131741524 |
| 200 | 40 | 4.749416351 | 600 | 90 | 10.27071738 |
| 200 | 50 | 6.37951088 | 600 | 100 | 11.41605854 |
| 200 | 60 | 7.12385726 | 800 | 10 | 1.063323736 |
| 200 | 70 | 9.355642796 | 800 | 20 | 2.049412966 |
| 200 | 80 | 10.4636941 | 800 | 30 | 3.172564507 |
| 200 | 90 | 11.81228352 | 800 | 40 | 4.201847792 |
| 200 | 100 | 14.11727881 | 800 | 50 | 5.46837163 |
| 400 | 10 | 1.117446184 | 800 | 60 | 6.651052237 |
| 400 | 20 | 2.501615286 | 800 | 70 | 7.739486456 |
| 400 | 30 | 3.522727013 | 800 | 80 | 8.862967968 |
| 400 | 40 | 4.428032398 | 800 | 90 | 9.946624041 |
| 400 | 50 | 5.699090719 | 800 | 100 | 11.67038965 |
| 400 | 60 | 7.1196661 | 1000 | 10 | 1.043431044 |
| 400 | 70 | 8.166855574 | 1000 | 20 | 2.064216852 |
| 400 | 80 | 9.625307083 | 1000 | 30 | 3.113944292 |
| 400 | 90 | 10.61369729 | 1000 | 40 | 4.184577942 |
| 400 | 100 | 12.83657336 | 1000 | 50 | 5.314064264 |
| 600 | 10 | 1.130707502 | 1000 | 60 | 6.67943573 |
| 600 | 20 | 2.127150536 | 1000 | 70 | 7.670101166 |
| 600 | 30 | 3.378539801 | 1000 | 80 | 8.791308641 |
| 600 | 40 | 4.376529217 | 1000 | 90 | 10.1499629 |
| 600 | 50 | 5.401717663 | 1000 | 100 | 11.64632177 |

# Experiment 4 (Varying K)

| k | time | processTime | threadTime |
|---|---|---|---|
| 1 | 0.6645600796 | 1.068286419 | 0.5724878311 |
| 2 | 0.6580889225 | 1.046080828 | 0.5700130463 |
| 3 | 0.6482551098 | 1.039890289 | 0.5900793076 |
| 4 | 0.3992986679 | 0.5947949886 | 0.3430445194 |
| 5 | 0.2745044231 | 0.3642506599 | 0.2360668182 |
| 6 | 0.1392707825 | 0.07155680656 | 0.09097719193 |
| 7 | 0.1357870102 | 0.06999707222 | 0.09005379677 |
| 8 | 0.1345300674 | 0.06937336922 | 0.09291887283 |
| 9 | 0.1344339848 | 0.06915354729 | 0.09139227867 |
| 10 | 0.1354603767 | 0.06906056404 | 0.0908164978 |